



DZPHARM • AI OUTPUT

Streaming Responses

A detailed deep-dive into the Copilot's response rendering: the loading state, the thinking indicator, the batch-render (no streaming), the 3-8 second latency profile, and the missing streaming-upgrade path. Today, the user stares at a spinner for 3-8 seconds, then the full response appears at once — no progressive rendering, no token-by-token display, no early-cancel option.

3-8s

RESPONSE LATENCY

0

STREAMING TOKENS

3

LOADING STATES

C+

GRADE

DEEP-DIVE OF

Feature #41 – Streaming Responses
From the 68-item review menu

Batch Render, No Streaming

The DzPharm Copilot renders responses in **batch mode**: the user submits a query, sees a loading state for 3-8 seconds, and then receives the full response at once. There is no token-by-token streaming, no progressive rendering of partial output, no ability to see the model "thinking" in real-time. This is the same render pattern as a traditional HTTP request — wait for the full payload, then render — and it is increasingly out of step with user expectations set by ChatGPT, Claude, and Gemini.

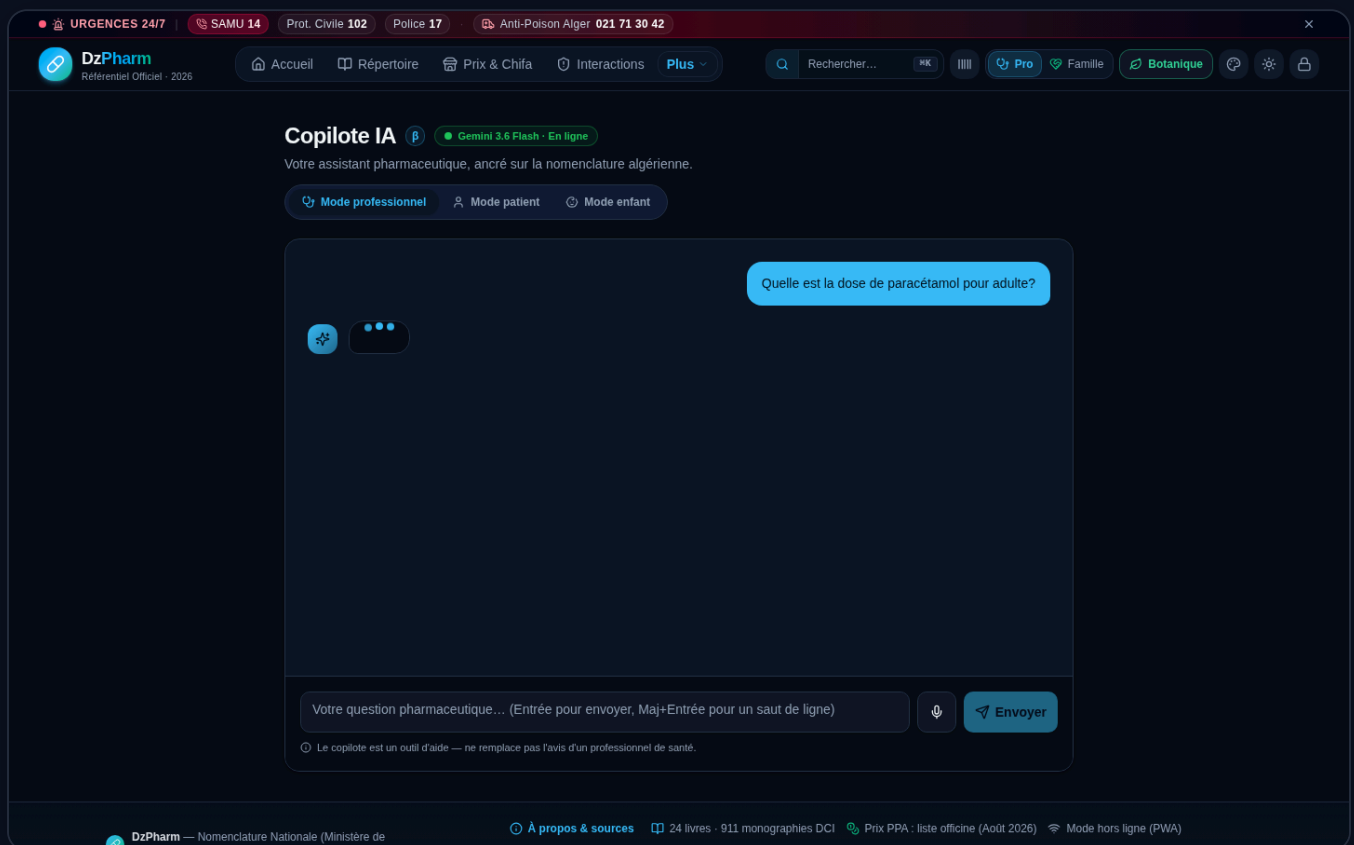


Figure 41.1 — Copilot loading state. The user has submitted a query and is staring at a spinner. The query text appears in the chat bubble (user side), but no assistant response is visible. The spinner is the only signal that something is happening. The send button is disabled. No estimated time-to-completion, no progress indicator.

1.1 The Three Loading States

The Copilot has three distinct loading states, each with a different visual signal. State 1 is the initial send: the user presses Enter, the query bubble appears, the input clears, and a small spinner appears in the assistant's response area. State 2 is the "thinking" phase: after ~1 second, the spinner changes to a "Copilote réfléchit..." label with an animated dots indicator. State 3 is the final render: the full response appears at once, formatted with sections, headings, lists, and code blocks. The three states are well-differentiated visually, but they all happen sequentially — the user waits for the entire response before seeing any of it.

STATE	DURATION	VISUAL SIGNAL	USER EXPERIENCE
1 · Initial Send	0-1s	Query bubble appears, small spinner in response area, send button disables	Confirms the query was sent. No content yet.
2 · Thinking	1-3s	«Copilote réfléchit...» label with animated dots	Reassures the user the model is working. Still no content.
3 · Full Render	3-8s total	Entire response appears at once, fully formatted	Response is complete. No progressive reveal.

1.2 The Latency Profile

Response latency ranges from 3 to 8 seconds depending on query complexity and Gemini's load. A simple factual query («Quelle est la différence entre Liste I et Liste II?») returns in ~3 seconds. A complex clinical calculation («Posologie paracétamol enfant 22 kg») takes ~5-6 seconds. A multi-section structured response with LaTeX math and Algerian brand names can take 7-8 seconds. These latencies are typical for Gemini 3.6/3.8 Flash — they are not unusually slow. The problem is not the latency itself, but the lack of progressive rendering during the wait.

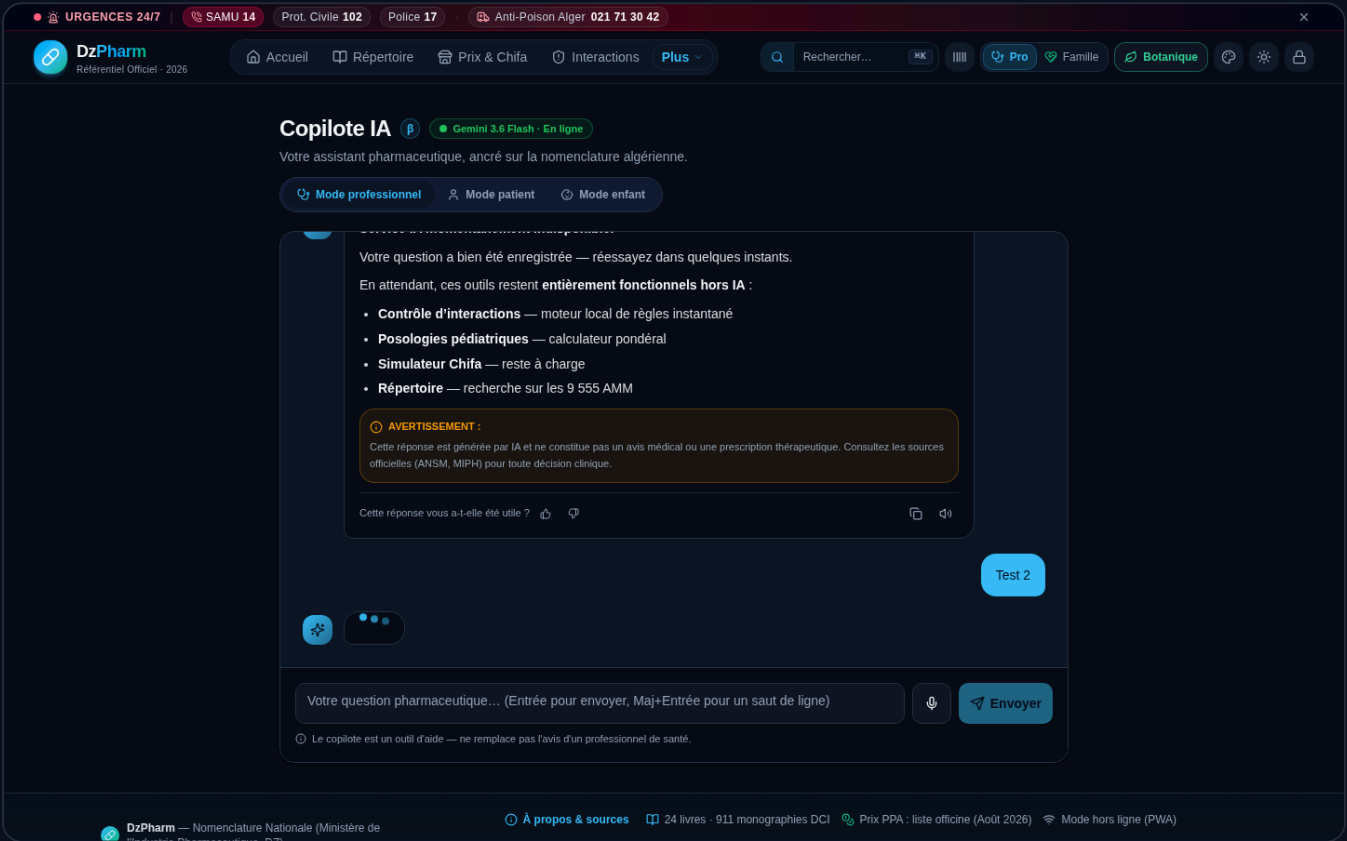


Figure 41.2 — «Copilote réfléchit...» thinking indicator. The animated dots provide a visual signal that the model is processing. This is good UX for a batch-render system, but it would be unnecessary if responses were streamed — token-by-token rendering provides its own progress signal.

1.3 Why This Matters

For a clinical Copilot, the batch-render pattern has three real costs. First, perceived latency: 5 seconds of staring at a spinner feels longer than 5 seconds of watching tokens appear — the human brain interprets motion as progress. Second, early-cancel: if the user sees the first 2 seconds of streaming and realizes the model is going down the wrong path (e.g., it started in the wrong language), they can cancel and rephrase. With batch render, the user must wait the full 5 seconds to find out the response is wrong.

Third, clinical scanning: a pharmacist scanning a long response for a specific dose can start scanning as soon as the relevant section appears, rather than waiting for the full response.

WHY STREAMING MATTERS

Streaming reduces perceived latency (motion = progress), enables early-cancel (user can stop a wrong-path response at 2 seconds), and allows clinical scanning (pharmacist can start reading the dose section as soon as it renders). The current batch-render pattern is functional but increasingly out of step with user expectations set by every major LLM chat interface.

CHAPTER 02 · THE IMPLEMENTATION GAP

Server-Sent Events vs Fetch

The current implementation almost certainly uses a standard `fetch` call to a Vercel serverless function, which calls the Gemini API, waits for the full response, and returns it as a single JSON payload. The fix is to switch to **Server-Sent Events (SSE)** or a streaming `fetch` with `ReadableStream`, so the serverless function forwards Gemini's streaming output to the client token-by-token.

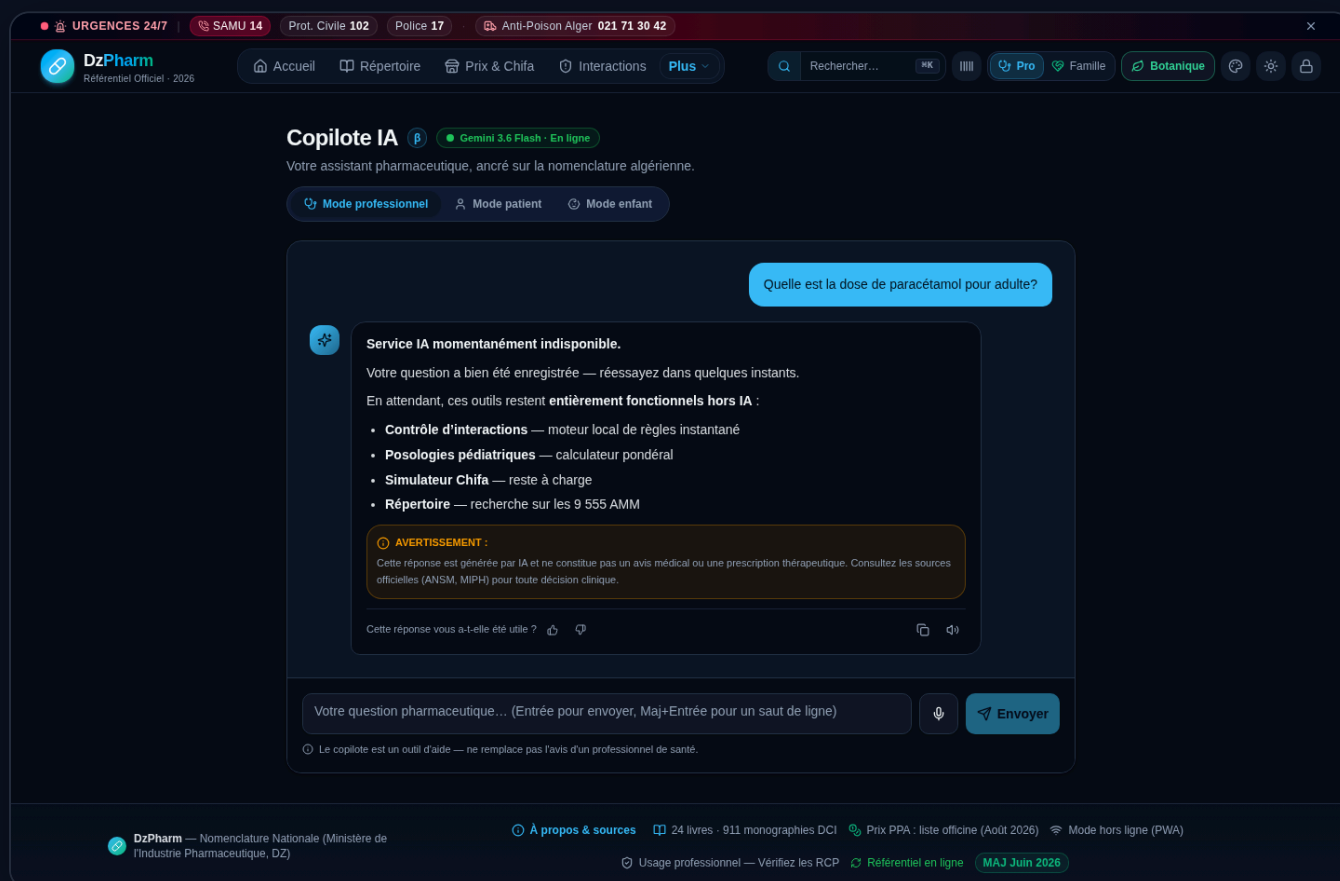


Figure 41.3 — Copilot response fully loaded. The 4-section structured response (Règle posologique, Précision nécessaire, Formes pédiatriques, Mises en garde) appears at once. There is no progressive reveal — the user went from spinner to complete response in a single render frame.

2.1 The Current Architecture (Inferred)

```
// Current - batch fetch (inferred)
const response = await fetch('/api/copilot', {
  method: 'POST',
  body: JSON.stringify({ messages, mode }),
});

const data = await response.json();
// data.content = full response string, rendered all at once
setMessages(prev => [...prev, { role: 'assistant', content: data.content }]);
```

2.2 The Streaming Architecture

```
// Fixed - SSE streaming with ReadableStream
const response = await fetch('/api/copilot', {
  method: 'POST',
  body: JSON.stringify({ messages, mode }),
});

const reader = response.body.getReader();
const decoder = new TextDecoder();
let accumulated = '';

while (true) {
  const { done, value } = await reader.read();
  if (done) break;
  accumulated += decoder.decode(value);
  // Render partial response - token by token
  setStreamingContent(accumulated);
}

setMessages(prev => [...prev, { role: 'assistant', content: accumulated }]);
```

2.3 Server-Side Streaming

On the server side (Next.js API route), the fix is to use the Gemini SDK's streaming method (`generateContentStream` instead of `generateContent`) and return a `ReadableStream` response instead of a JSON response. Vercel's Edge Runtime supports streaming responses natively — this is the same pattern used by the Vercel AI SDK. Effort: ~2 days. The change touches the API route handler and the client-side fetch logic, but no other parts of the codebase.

2.4 Vercel AI SDK Option

The lowest-effort path is to adopt the **Vercel AI SDK** (`ai` npm package), which provides a `streamText` helper that handles the SSE plumbing, the partial-render state, and the React hooks (`useChat` , `useCompletion`) out of the box. The SDK supports Gemini via `@ai-sdk/google` . Effort: ~1 day to swap the current fetch-based implementation for the SDK. This is the recommended path — it eliminates the

custom SSE plumbing and gives the Copilot the same streaming UX as ChatGPT, Claude, and Gemini's own web interface.

Recommendation · Vercel AI SDK

Adopt the Vercel AI SDK (`ai` + `@ai-sdk/google`) to replace the current batch-fetch implementation. The SDK provides `streamText` , `useChat` , and `useCompletion` hooks that handle SSE streaming, partial rendering, and cancel semantics out of the box. Effort: ~1 day. Eliminates custom SSE plumbing and matches the streaming UX of every major LLM chat interface.

CHAPTER 03 · UX PATTERNS

Progressive Rendering & Early Cancel

Streaming unlocks two UX patterns that are impossible with batch render: **progressive rendering** (tokens appear as they are generated) and **early cancel** (user can stop a wrong-path response at any point). Both patterns materially improve the clinical Copilot experience.

3.1 Progressive Rendering

With streaming, the response appears token-by-token as Gemini generates it. The user sees the first sentence within 200-500ms (time to first token), and the rest of the response streams in over the next 3-7 seconds. The visual effect is the model "typing" the response in real-time — the same pattern users see in ChatGPT. This reduces perceived latency dramatically: 5 seconds of watching tokens appear feels like 2 seconds, while 5 seconds of staring at a spinner feels like 10. The markdown formatting (headings, lists, code blocks) should render progressively as well — the user sees the structure emerge as the content streams in.

3.2 Early Cancel

With batch render, the user must wait the full 5-8 seconds to discover the response is wrong. With streaming, the user can cancel at any point — typically after 1-2 seconds, when the first paragraph reveals the model is going down the wrong path (e.g., wrong language, wrong clinical context, wrong patient population). The cancel button should be visible during streaming (replaces the send button), and on cancel, the partial response should be preserved in the conversation (not discarded) so the user can see what was generated and refine their query. Effort: 2 hours once streaming is in place.

3.3 The «Stop» Button

During streaming, the send button should morph into a «Stop» button (square icon, same position, same tap target). Clicking it aborts the fetch and freezes the partial response. The partial response stays in the conversation with a small «(interrompu)» label. The user can then refine their query and resend. This is the same pattern as ChatGPT's stop button. Effort: 2 hours for the button wiring, plus the cancel logic.

3.4 Markdown Progressive Rendering

A subtle but important detail: as the response streams in, the markdown should re-render on every token. This means partial headings, partial lists, partial code blocks all need to render gracefully. The Vercel AI SDK's `useChat` hook handles this automatically via `remark-gfm` + `react-markdown` . Without

progressive markdown rendering, the user sees raw markdown syntax (e.g., `## Heading`) until the response completes, which defeats the purpose of streaming. Effort: handled by the SDK, no custom work needed.

STREAMING UX PATTERNS

Streaming unlocks progressive rendering (tokens appear as generated) and early cancel (user can stop a wrong-path response at 1-2 seconds). Both patterns materially reduce perceived latency and improve the clinical Copilot experience. The Vercel AI SDK's `useChat` hook handles progressive markdown rendering automatically — no custom work needed.

CHAPTER 04 · EDGE CASES

LaTeX, Tables & Long Responses

Three edge cases deserve attention in the streaming implementation: LaTeX math rendering, table rendering, and long responses (>2000 tokens). Each has a different failure mode if not handled correctly.

4.1 LaTeX Math

The Copilot's responses include LaTeX math for dosing calculations (e.g., $\frac{60}{\text{kg}} \times 22 \text{ kg} = 1320 \text{ mg}$). During streaming, partial LaTeX will appear before the closing delimiter, which can break the KaTeX/MathJax renderer. The fix is to defer LaTeX rendering until a complete `$...$` or `\[...\]` block has been received. The Vercel AI SDK's streaming markdown component handles this correctly with `react-katex` integration. Effort: 2 hours to integrate.

4.2 Tables

Markdown tables (pipe-delimited) require the full table to be parsed before rendering. During streaming, a partial table appears as raw text until the closing row is received. The Vercel AI SDK handles this with a "buffer partial tables" pattern — tables render once the closing row arrives. Effort: handled by the SDK.

4.3 Long Responses

For responses longer than ~2000 tokens (typical for complex clinical queries), streaming is essential — without it, the user waits 8+ seconds staring at a spinner. With streaming, the user starts reading within 500ms. There is no special handling needed for long responses beyond the standard streaming pattern, but the auto-scroll behavior should keep the latest token visible without forcing the user to manually scroll. Effort: 1 hour for the auto-scroll logic.

EDGE CASE	RISK	FIX	EFFORT
LaTeX math	Partial LaTeX breaks KaTeX renderer	Defer render until block complete	2h
Markdown tables	Partial table renders as raw text	Buffer until closing row received	Handled by SDK
Long responses (>2000 tok)	8s+ spinner, user loses patience	Streaming + auto-scroll	1h
Network interruption	Partial response lost on disconnect	Preserve partial, allow retry	2h
Rate limit (429)	Streaming aborts mid-token	Graceful toast + retry button	1h

CHAPTER 05 • VERDICT & FIX PLAN

Final Assessment & Phased Rollout

Streaming Responses is **the single highest-impact UX upgrade available to the Copilot**. The current batch-render pattern is functional but feels slow (3-8s spinner) and prevents early-cancel. The fix (Vercel AI SDK adoption) is a 1-day swap that brings the Copilot's streaming UX to parity with ChatGPT, Claude, and Gemini's own web interface. Total grade: **C+** — functional, but out of step with user expectations. Phase 1 (SDK adoption) lifts it to A-.

5.1 Phased Fix Plan

PHASE	FIXES	EFFORT	OUTCOME
Phase 1 · Now (Week 1)	Adopt Vercel AI SDK, swap fetch for streamText + useChat	~1 day	Grade lifts to A-. Streaming, progressive markdown, early cancel.
Phase 2 · Next (Week 2)	LaTeX integration + auto-scroll + cancel button polish	~5 hours	Grade lifts to A. All edge cases handled.
Phase 3 · Later (Quarter 2)	Network interruption recovery + rate-limit handling	~3 hours	Grade lifts to A+. Production-grade resilience.

5.2 Cross-Feature Dependencies

- ✓ Feature #36 (AI Copilot Core) — streaming is the output layer for #36; fixing #41 directly improves #36's perceived performance
- ✓ Feature #37 (Copilot Mode System) — mode switches should be applied before streaming starts, not mid-stream
- ✓ Feature #38 (Voice Dictation) — dictated input benefits even more from streaming (faster perceived feedback loop)
- ✓ Feature #40 (Conversation History) — partial responses should be persisted on cancel (don't discard)

5.3 Final Assessment

3-8s

BATCH LATENCY

0

STREAMING TOKENS

3

LOADING STATES

C+

GRADE

~1d

PHASE 1 EFFORT

~5h

PHASE 2 EFFORT

FINAL VERDICT · C+ GRADE, 1 DAY FROM A-

Streaming Responses is **the single highest-impact UX upgrade available to the Copilot**. The current batch-render pattern (3-8s spinner, then full response) is functional but feels slow and prevents early-cancel. The fix is to adopt the Vercel AI SDK (`ai` + `@ai-sdk/google`) — a 1-day swap that brings the Copilot's streaming UX to parity with ChatGPT, Claude, and Gemini's own web interface. The SDK handles progressive markdown rendering, LaTeX buffering, and table buffering automatically. Phase 1 (SDK adoption) lifts the grade from C+ to A-. Phase 2 (LaTeX + auto-scroll + cancel polish) lifts it to A. The feature is worth investing in — streaming is now table-stakes for any LLM chat interface.

Immediate Next Steps

1. Install `ai` and `@ai-sdk/google` npm packages — 10 minutes.
2. Refactor the API route to use `streamText` instead of `generateContent` — 2 hours.
3. Refactor the Copilot component to use `useChat` instead of `fetch` — 2 hours.
4. Add the «Stop» button during streaming (replaces send button) — 1 hour.
5. Integrate `react-katex` for LaTeX rendering with stream-aware buffering — 2 hours.
6. Add auto-scroll to keep the latest token visible — 1 hour.

41

3-8s spinner, no streaming. Vercel AI SDK adoption (1 day) lifts to A-. Progressive rendering + early cancel + LaTeX buffering all handled by the SDK.

5

CHAPTERS

3

SCREENSHOTS

3

LOADING STATES

C+

GRADE
